RITAL : Chirac ou Mitterrand ? Comm' positif ou négatif ?

Etudiant : **Nils Barrellon 21401602**

Github associé : https://github.com/nbarrellon/RITAL2026

1 Formulation du problème

Chirac ou Mitterrand : on dispose de la transcription d'un débat entre les deux présidents de la République, MM Chirac et Mitterrand. Chaque phrase prononcée par l'un ou l'autre est labellisée. On attribue le label M pour celles proposées par M.Mitterrand que l'on convertit en -1 et le label C (converti en 1) si c'est M.Chirac qui parle.
Les phrases labellisées de ce débat constitueront donc les données d'entraînement.
L'objectif du projet est de pouvoir attribuer à son auteur une phrase quelconque prononcée dans un autre contexte : on parle de **diarisation du locuteur**.

Commentaire positif ou négatif : la dataset est constituée d'une banque de commentaires autour de différents films. L'objectif est de décider si tel ou tel commentaire est un commentaire positif ou négatif : on parle ici d'**analyse de sentiment**.

2 Etude du problème de diarisation du locuteur

2.1 Bag of words

J'ai créé différents bag of words (*bog*) pour les documents afin d'en tester la pertinence.

2.1.1 Pré-processing

J'effectue en amont de la construction du *bog* un nettoyage du texte. Les options retenues sont :

- suppression de la ponctuation ;
- suppression des chiffres ;
- mise en minuscule ;

Ce nettoyage est opéré par ma fonction personnalisée de pré-processing suivante :

```
import codecs
import re
import string
from unidecode import unidecode

def preprocessPerso(text):
    #suppression de la ponctuation
    punc = string.punctuation
    punc += '\n\r\t'
    text = text.translate(str.maketrans(punc, ' '*len(punc)))
    #suppression des chiffres
    text = re.sub(r"[0-9]","",text)
```

```
    #suppression des accents
    text = unidecode(text)
    #mise en minuscule
    return text.lower()
```

2.1.2 Stopwords

Je supprime les mots blancs avant construction du *bog*. La liste des stopwords est obtenue par concaténation des stopwords français donnés par la bibliothèque `nltk` auxquels j'ajoute une liste personnelle, établie à partir de la liste des mots les plus présents et non-discriminants.

```
# Mots blancs + mots les plus fréquents

nltk.download('stopwords')
from nltk.corpus import stopwords
final_stopwords_list = stopwords.words('french')
#mots les plus fréquents donc non-discriminants
final_stopwords_list += ['ai','au','aujourd','aussi','autres','aux','
   avec','avez' ,'avons',
 'bien', 'ce' ,'cela', 'ces', 'cette' ,'ceux', 'chacun' ,'comme', '
    dans' ,'date'
 'de', 'depuis', 'des' ,'deux' ,'dire' ,'doit' ,'dont' ,'du'
 'elle' ,'en' ,'encore', 'ensemble', 'entre', 'est' ,'et', 'etat', '
    ete' ,'etre',
 'europe' ,'faire' ,'fait' ,'faut', 'francais' ,'france', 'hui' ,'ici'
    , 'il' ,'ils',
 'je' ,'la', 'le' ,'les', 'leur' ,'leurs', 'mais', 'meme' ,'monde', '
    monsieur' ,'ne',
 'nom', 'nos', 'notre', 'nous', 'on', 'ont' ,'ou', 'paix', 'par', 'pas
    ', 'pays',
 'peut' ,'plus' ,'politique' ,'pour', 'president' ,'qu', 'que' ,'qui',
     'sa', 'sans',
 'se' ,'ses' ,'si' ,'son' ,'sont', 'sur', 'temps', 'tous', 'tout', '
    toute' ,'toutes',
 'tres', 'un' ,'une' ,'union', 'vie', 'vos' ,'votre', 'vous']
```

Cette liste a été obtenue à l'aide du script ci-dessous :

```
from sklearn.feature_extraction.text import TfidfVectorizer

use_idf=True
smooth_idf=True
sublinear_tf=False

vectorizer = TfidfVectorizer(preprocessor=preprocessPerso,use_idf=
   use_idf, smooth_idf=smooth_idf, sublinear_tf=sublinear_tf,
```

```
    max_features=100)
BOW1 = vectorizer.fit_transform(alltxts)
vocabulaire = vectorizer.get_feature_names_out()
```

2.1.3 Premier bog

Bag of words simple. Taille du vocabulaire = 26894

```
vectorizer1 = CountVectorizer(stop_words=final_stopwords_list,analyzer
    ="word", preprocessor=preprocessPerso)
BOW1 = vectorizer1.fit_transform(alltxts)
```

2.1.4 Deuxième bog

Utilisation de TF-IDF. Nombre de mots du vocabulaire : 2000

```
use_idf=True
smooth_idf=True
sublinear_tf=False

vectorizer2 = TfidfVectorizer(use_idf= use_idf, smooth_idf=smooth_idf
    ,\
sublinear_tf=sublinear_tf,                          preprocessor=
    preprocessPerso,stop_words=final_stopwords_list,max_features=2000)
BOW2 = vectorizer2.fit_transform(alltxts)
```

être
où
date
français
président
très
développement
avenir
histoire
beaucoup
toujours
économique
abord
notamment
elles
société
messieurs
sécurité
place
ans
enfin
liberté
jeunes
grande
confiance
entreprises
européenne
solidarité
déjà
volonté
également
grand
mesdames
oeuvre
état
naturellement
esprit
sociale
croissance
afrique
sûr
trop
pense
là
mieux
respect
sens
part
quand
contre
non
chaque
fois
autre
années
moyens
hommes
nouvelles
culture
démocratie
république
nouvelle
souhaitesouvent
travail
alors
service
ainsi
jamais
coeur
grands
après
progrès
avoir
dialogue
quelques
année
premier
action
parce
droit
donc
cet
emploi
doivent
coopération
communauté
devons
celle
nation
peuples
première
amitié
moyens
gouvernement
sous
naturellement
voudrais
besoin
peu
forte
partenaires
nouveau
grandes
croissance
ailleurs
vrai
nations
avant
essentiel
afrique
sûr
pense
là
mieux
sens
respect
ans
liberté
jeunes
enfin
fois
autre
années
hommes
nouvelles
culture
démocratie
république

```
nlp = spacy.load("fr_core_news_sm", disable=["parser", "ner"])

def lemmatisation(text, stopwords):
    text = preprocessPerso(text)
    doc = nlp(text.lower())
    # Lemmatiser uniquement les mots qui ne sont pas des stopwords
    lemmatized = [token.lemma_ for token in doc if token.text not in
        stopwords]
    text = " ".join(lemmatized).strip()
    #on supprime les espaces surnuméraires créés par la racinisation
    text = re.sub(r'\s+', ' ', text)
    return text

# Pré-traiter tous les textes
alltxts_lemmatise = []
for text in alltxts:
    alltxts_lemmatise.append(lemmatisation(text, final_stopwords_list)
        )
```

Le résultat est très probant comme le montre l'exemple ci-dessous (1re phrase du débat avant et après racinisation) :

```
C'est toujours très émouvant de venir en Afrique car c'est probablement
l'une des rares terres du monde où l'on ait conservé cette convivialité,
cette amitié, ce respect de l'autre qui s'expriment avec chaleur, avec spontanéité
et qui réchauffent le coeur de ceux qui arrivent et de ceux qui reçoivent.

-------------------------------
être toujours tre emouver venir afrique car être probablement rare terre avoir
conserve convivialite amitie respect autre exprimer chaleur spontaneite rechauffer
coeur celui arriver celui recoiver
```

Taille du vocabulaire = 19049

Je note que, sans surprise, les auxiliaires **être** et **avoir** arrivent en tête des mots les plus fréquents.
Je décide donc de les ajouter aux mots blancs.
A noter : ce processus de racinisation est très long et j'ai donc sauvegardé le vocabulaire obtenu pour ne pas avoir à le recalculer à chaque fois.

```
with open("corpus_nettoye_lemmatise.txt","w",encoding="utf-8") as f:
    for i in range(len(alltxts_lemmatise)):
        f.write(alltxts_lemmatise[i])
        f.write("\n")
```

2.1.7 Cinquième bog

Je conjugue bigrammes et racinisation. Taille du vocabulaire = 345610

```
ngram_range = (1,2)
final_stopwords_list += ['avoir',"être"]
vectorizer5 = CountVectorizer(ngram_range=ngram_range,analyzer='word'
    ,\
stop_words=final_stopwords_list,preprocessor=preprocessPerso)
BOW5 = vectorizer5.fit_transform(alltxts_lemmatise)
```

Je note que les mots les plus fréquents sont des monogrammes.

2.1.8 Sixième bog

Je construis un bog binaire. Taille du vocabulaire = 10000

```
min_df=5
max_df=0.5
max_features=10000
vectorizer6 = CountVectorizer(max_df=max_df,min_df=min_df,max_features
   =max_features,\
binary=True,stop_words=final_stopwords_list)
BOW6 = vectorizer6.fit_transform(alltxts)
```

2.1.9 Septième bog

J'applique une racinisation avant un TF-IDF. Taille du vocabulaire = 4000

```
use_idf=True
smooth_idf=True
sublinear_tf=False
vectorizer7 = TfidfVectorizer(use_idf= use_idf, smooth_idf=smooth_idf,
    sublinear_tf=sublinear_tf,\
                              stop_words=final_stopwords_list,
                                 max_features=4000)
BOW7 = vectorizer7.fit_transform(alltxts_lemmatise)
```

2.1.10 Huitième bog

J'applique une racinisation avant un TF-IDF sur des bigrammes. Taille du vocabulaire = 4000

```
use_idf=True
smooth_idf=True
sublinear_tf=False
vectorizer8 = TfidfVectorizer(use_idf= use_idf, smooth_idf=smooth_idf,
    sublinear_tf=sublinear_tf,\
                              stop_words=final_stopwords_list,
                                 max_features=4000)
BOW8 = vectorizer8.fit_transform(alltxts_lemmatise)
```

2.1.11 Neuvième bog

Je me suis aperçu, en séparant les données par locuteur en 2 corpus distincts, que M.Chirac a beaucoup plus parlé que M.Mitterrand lors de ce débat.

```
corpus1 = [alltxts[i] for i in range(len(alltxts)) if alllabs[i]==1] #
   Chirac
corpus2 = [alltxts[i] for i in range(len(alltxts)) if alllabs[i]==-1]
   #Mitterand

#Taille des corpus
print("------------------Taille des corpus-----------------------")
print("Chirac:",len(corpus1))
print("Mitterrand",len(corpus2))
```

```
------------------Taille des corpus-----------------------
Chirac: 49890
Mitterrand 7523
```

Ceci a une influence considérable sur l'entrainement et les prédictions des différents modéles. La métrique de la précision, très bonne quelque soit le *bog* utilisé, n'est plus pertinente.

Il m'a donc semblé pertinent de ré-équilibrer les données. J'ai donc créé un neuvième *bog* en ramenant le nombre d'interventions de M.Chirac à un nombre égal à celui de M.Mitterrand.

```
corpus1 = [(alltxts_lemm[i],alllabs[i]) for i in range(len(alltxts))
   if alllabs[i]==1] #Chirac
corpus2 = [(alltxts_lemm[i],alllabs[i]) for i in range(len(alltxts))
   if alllabs[i]==-1] #Mitterand

corpus = corpus2+corpus1[:len(corpus1)//6]
new_alltxts = [c[0] for c in corpus]
new_labels = [[c[1] for c in corpus]]

ngram_range = (1,2) # unigrams and bigrams
```

```
use_idf=True
smooth_idf=True
sublinear_tf=False

vectorizer9 = TfidfVectorizer(ngram_range=ngram_range,analyzer="word",
   use_idf= use_idf, smooth_idf=smooth_idf, sublinear_tf=sublinear_tf
   ,\
                              stop_words=final_stopwords_list,
                                 max_features=5000)
BOW9 = vectorizer9.fit_transform(new_alltxts)
```

Taille du vocabulaire = 5000

3 Classificateurs

J'applique trois méthodes de classification sur mes différents *bog* :

- Naïve Bayes
- Regression logique
- SVD

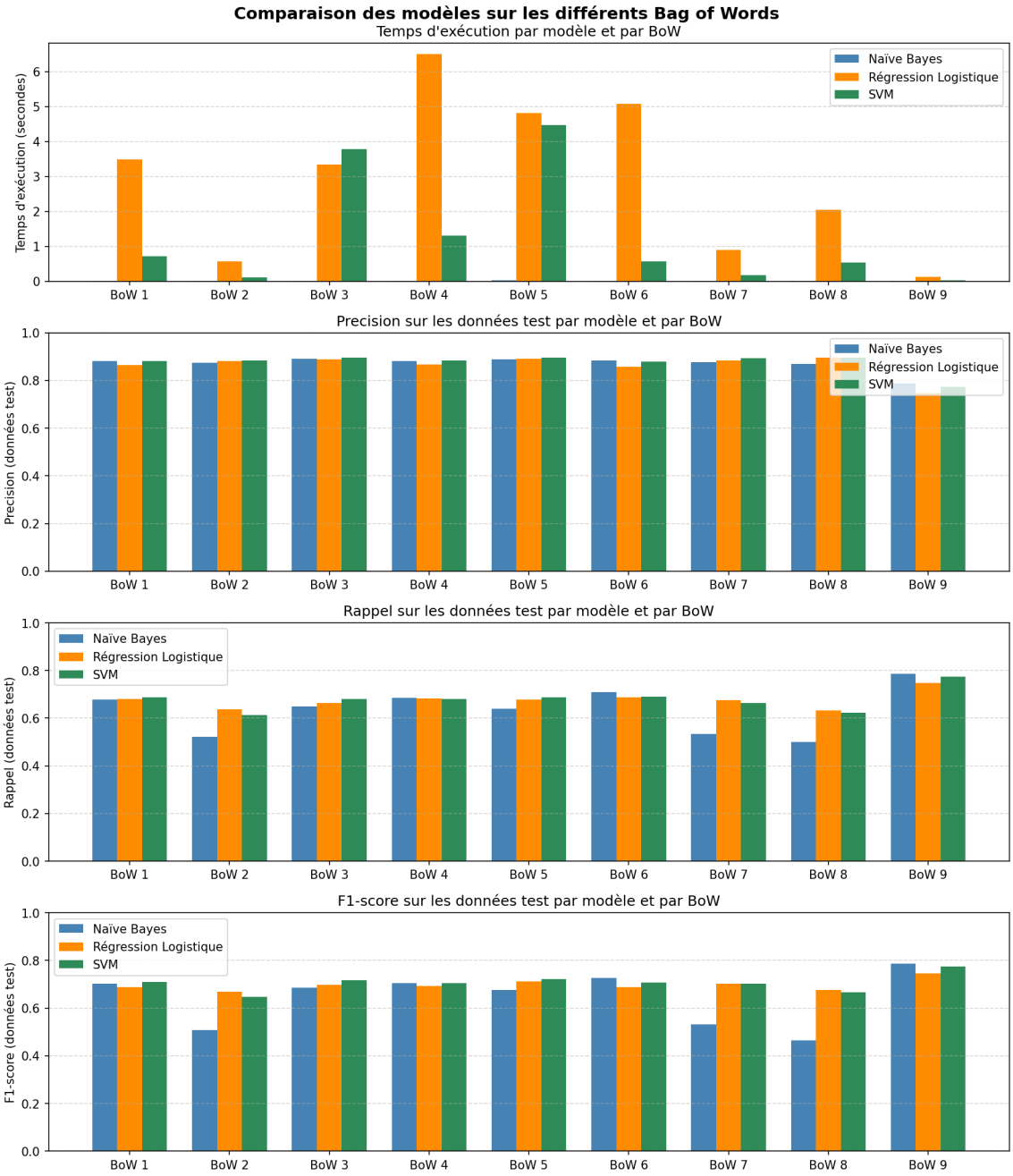
La méthode **Naïve Bayes** applique le théorème de Bayes en supposant que les mots sont indépendants entre eux (d'où le "naïf"). Pour un texte, il calcule la probabilité que chaque mot appartienne à chaque classe, puis multiplie ces probabilités pour classer le document.

La **régression logistique** modélise la probabilité d'appartenir à une classe via une fonction sigmoïde appliquée à une combinaison linéaire des mots. C'est un modèle linéaire qui une frontière de décision droite entre les classes.

La méthode **SVM pour Support Vector Machine** cherche l'hyperplan qui sépare les classes avec la marge maximale entre les points les plus proches de chaque classe (les vecteurs de support). Grâce à l'astuce du kernel, il peut projeter les données dans un espace de dimension supérieure pour séparer des classes non linéairement séparables.

4 Métriques

J'ai testé les 3 méthodes de classification sur chacun des *bog* et mesuré 3 métriques différentes sur les données test : la précision, le rappel et le f1-score. La figure montre de plus le temps d'exécution des 3 modèles pour chaque *bog*.



Le ré-équilibrage des données semble mener à un résultat de meilleure qualité. J'ai donc testé ce *bog* pour ma première soumission de données sur la plateforme.
Voici le script que j'ai utilisé pour générer mon fichier de résultats :

```
#Choix du bog
vectorizer = vectorizer9

#Application du BOG sur les données à tester
test_nouvelles_phrases = vectorizer.transform(nouvelles_phrases)
```

```
X_train = BOW9
y_train = new_labels

#Choix du modèle
model = LogisticRegression(random_state=0, solver='liblinear',
   max_iter=10
00, tol=1e-8, C=100.0)

# Entrainement du modèle
model.fit(X_train, y_train)


probabilites = model.predict_proba(test_nouvelles_phrases)[:, 1]  #
   Probabilité classe "Chirac"

# Créer un DataFrame avec les probabilités
df = pd.DataFrame({
    "phrase": nouvelles_phrases,
    "probabilite_chirac": 1-probabilites #car on veut la proba pour
       Mitterand
})

# Écrire dans un fichier CSV
df.to_csv("probabilites_chirac.csv", index=False, header=False,columns
   =["probabilite_chirac"])
```

Cette première soumission m'a rendu les métriques suivantes :

- f1 =35.052
- auc = 70.976
- ap = 29.524

Ces résultats m'ont énormément surpris puisque très différents de ceux obtenus sur les données test. J'ai donc cru que je m'étais trompé de probabilité et ai soumis un nouveau fichier csv contenant `1-proba`. Les résultats étaient pires. Comment expliquer ces mauvais scores ?
J'ai dans l'idée que la normalisation était inadaptée : la racinisation pourrait bien avoir supprimé des informations utiles. J'ai donc soumis les données obtenues avec le premier *bog*.
Cette deuxième soumission m'a rendu les métriques suivantes :

- f1 =15.463
- auc = 29.817
- ap = 9.285

Résultats catastrophiques et incompréhensibles. Une nouvelle fois, j'ai pensé avoir inversé les probabilités mais, après avoir soumis les probabilités inversées les résultats n'étaient pas meilleurs. Je suis donc revenu aux "basiques" pour ma cinquième soumission. J'ai utilisé un *bog* TF-IDF, avec utilisation des stopwords fournis par la bibliothèque `nltk` avec mon nettoyage en amont personnel.

```
from nltk.corpus import stopwords
final_stopwords_list = stopwords.words('french')
use_idf=True
smooth_idf=True
sublinear_tf=False

vectorizer10 = TfidfVectorizer(ngram_range=(1,2),analyzer="word",
   use_idf= use_idf, smooth_idf=smooth_idf, sublinear_tf=sublinear_tf
   ,\
                              stop_words=final_stopwords_list,
                                 preprocessor=preprocessPerso)
BOW10 = vectorizer10.fit_transform(alltxts)
```

Cette cinquième soumission m'a rendu les métriques suivantes :

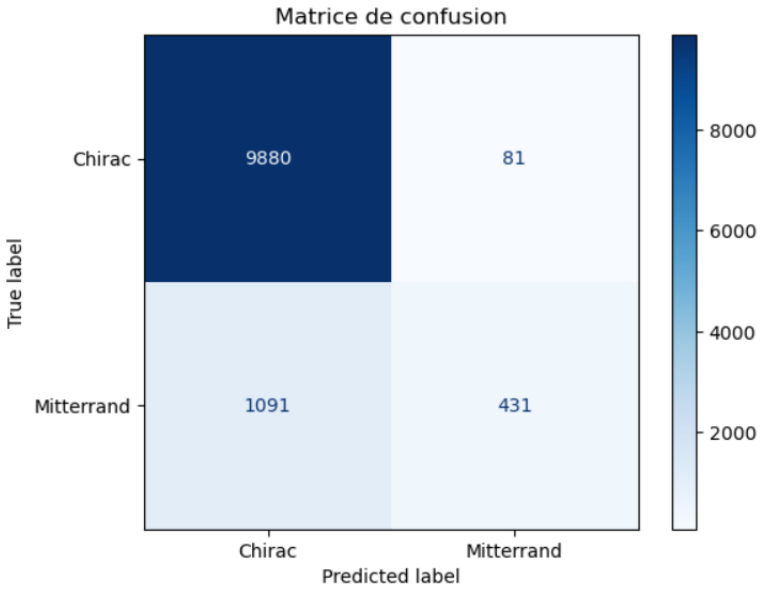
- f1 =56.392
- auc = 87.603
- ap = 63.694

Résultats meilleurs mais toujours pas satisfaisants. Mon nombre d'essais se rabougrissant comme peau de chagrin, j'ai procédé à une analyse plus poussée en amont de ma prochaine soumission. Notamment en établissant la matrice de confusion de mes prédictions obtenues avec mon dernier *bog*.

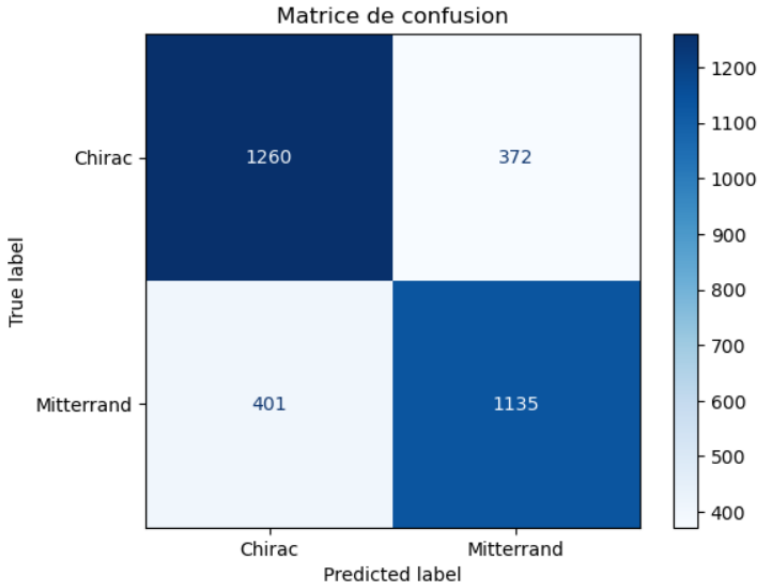
```
[X_train, X_test, y_train, y_test]  = train_test_split(BOW10, alllabs,
    test_size=0.2, shuffle=True)


# Entrainement du modèle
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

#Convertir les labels en 'N' (négatif) ou 'P' (positif)
y_pred_labels = ['M' if lbl <0.5 else 'C' for lbl in y_pred]
y_test_labels = ['M' if lbl==-1 else 'C' for lbl in y_test]
# Matrice de confusion
cm = confusion_matrix(y_test_labels, y_pred_labels)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["
   Chirac","Mitterrand"])
```

```
disp.plot(cmap=plt.cm.Blues)
plt.title("Matrice de confusion")
plt.show()
```

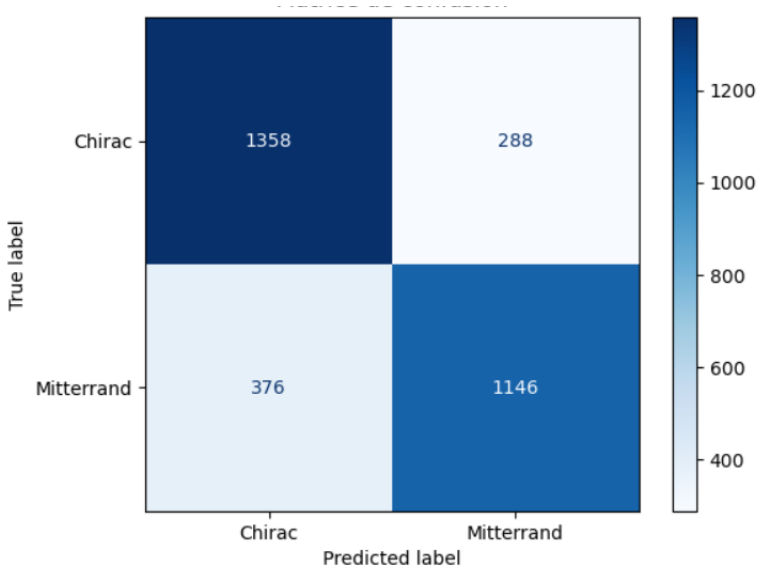
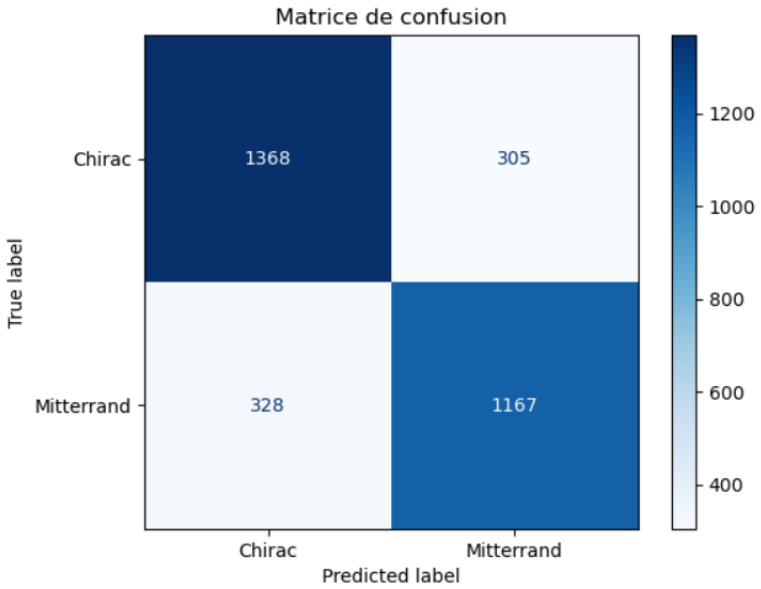
J'ai obtenu un résultat surprenant :



Le locuteur Chirac est très bien discriminé. En revanche, l'erreur est la norme pour le locuteur Mitterrand. Ceci allait dans le sens de mes premières observations : M.Chirac a beaucoup plus parlé que M.Mitterrand et le modèle est biaisé. Ce qui ressort sur la matrice de confusion, réalisé après ré-équilibrage des données brutes.



J'ai noté une amélioration si je ne restreins pas la taille du vocabulaire...



Enfin, j'ai renouvellé l'opération de lemmatisation sur les données ré-équilibrées. La matrice de confusion est légèrement meilleure.